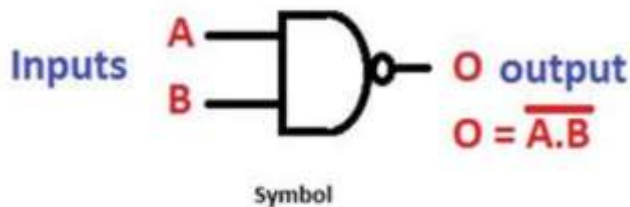


Lec. 4

NAND Gate



Inputs		Output
A	B	O
0	0	1
0	1	1
1	0	1
1	1	0

Truth table

From the diagram, the NAND gate is only 0 if both inputs are ones.

From $(w_1 \cdot A + w_2 \cdot B) + b$, initializing w_1, w_2 as 1 and b as -1, we get $(1)A + (1)B - 1$.

ROW 1:

Passing the first row of the AND logic table ($A=0, B=0$), we get $0 + 0 - 1 = -1$. From the Perceptron rule, if $wx + b \leq 0$, then $y = 0$. This row is incorrect, as the output is 1 for the NAND gate. So, we want value that will make inputs $A=0, B=0$ give y a value of 1. If we change b to 1, we have $0 + 0 + 1 = 1$. From the Perceptron rule, this works.

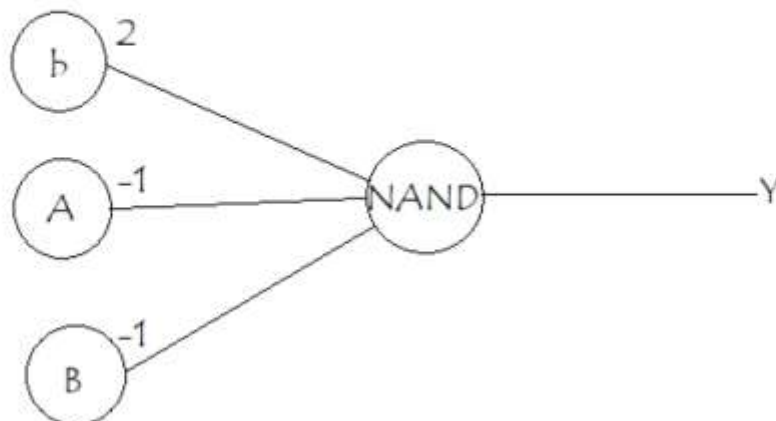
ROW 2:

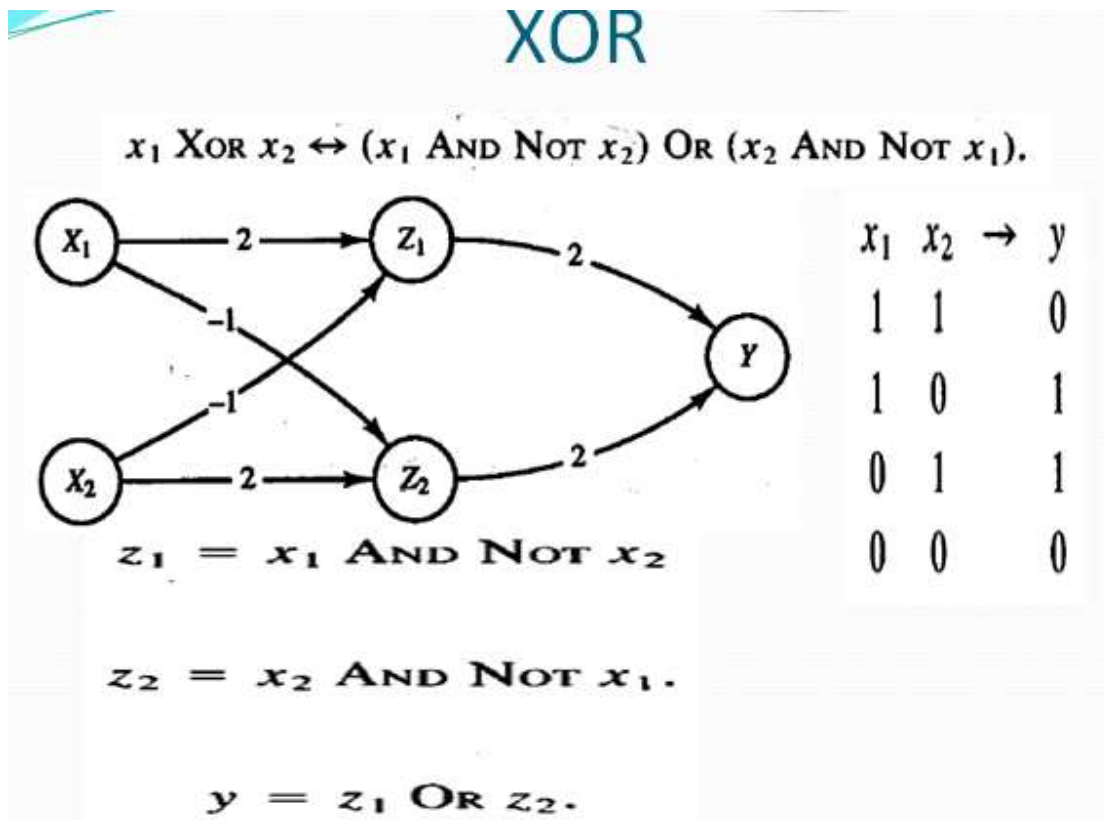
Passing ($A=0, B=1$) we get $0 + 1 + 1 = 2$. From the Perceptron rule, if $wx + b > 0$ then $y = 1$. This row is also correct for both row 2 and row 3.

ROW 4:

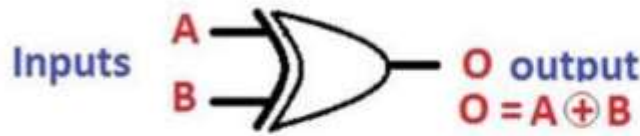
Passing ($A=1, B=1$) we get $1 + 1 + 1 = 3$. This is not the expected output, as the output is 0 for a NAND combination of $A=1$ and $B=1$. Changing the values of w_1 and w_2 to -1, and value of b to 2, we get $-1 - 1 + 2 = 0$. It works for all rows.

Therefore, we can conclude that the model to achieve a NAND gate, using Perceptron algorithm is: $-A - B + 2$.





XOR Gate



Symbol

Inputs		Output
A	B	O
0	0	0
0	1	1
1	0	1
1	1	0

Truth table

The Boolean representation of an XOR gate is

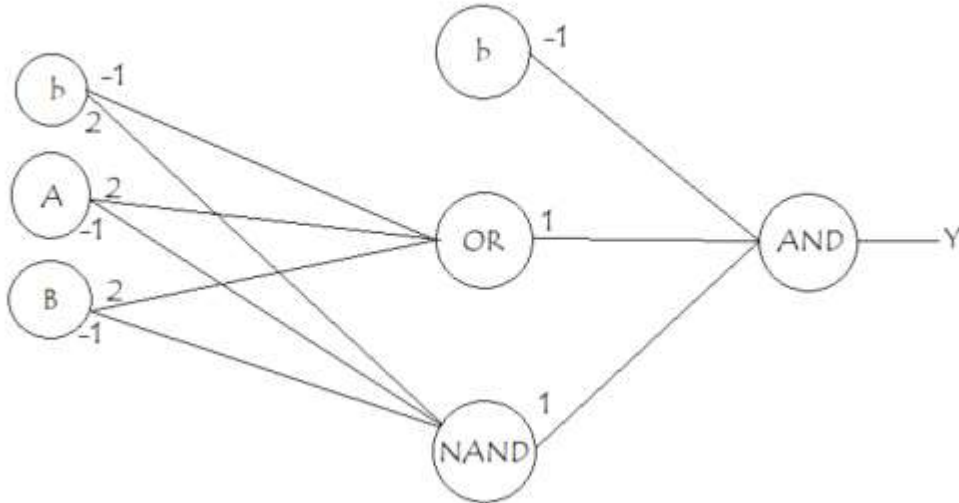
$$AB + \bar{A}B$$

We first simplify the Boolean expression

$$\begin{aligned} &AB + \bar{A}B + A\bar{A} + \bar{B}B \\ &AB + (\bar{A}B + A\bar{A}) + \bar{B}B \\ &A(\bar{A} + \bar{B}) + B(\bar{A} + \bar{B}) \\ &(A + B)(\bar{A} + \bar{B}) \\ &(A + B)(\overline{AB}) \end{aligned}$$

From simplified expression, we can say that the XOR gate consists of an OR gate, a NAND gate, and an AND gate:

- AND ($A + B - 1$)
- OR ($2A + 2B - 1$)
- NAND ($-A - B + 2$)



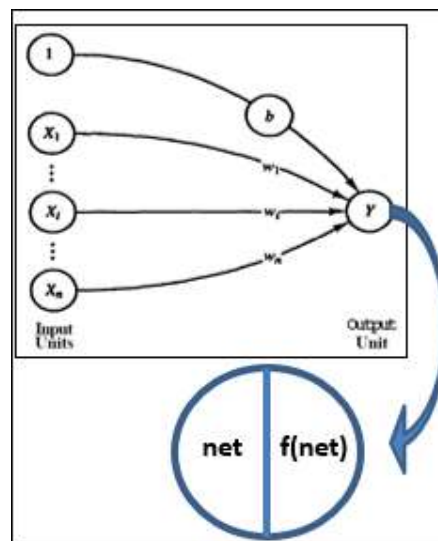
Hebb Learning

- 1- Architecture
- 2- Algorithm
- 3- Application / logic gate

Hebb learning

Donald Hebb, a psychologist at McGill University, designed the first learning law for artificial neural networks [Hebb, 1949]. His premise was that if two neurons were active simultaneously, then the strength of the connection between them should be increased. Refinements were subsequently made to this rather general statement to allow computer simulations [Rochester, Holland, Haibt & Duda, 1956]. The idea is closely related to the correlation matrix learning developed by Kohonen (1972) and Anderson (1972) among others. An expanded form of Hebb learning [McClelland & Rumelhart, 1988] in which units that are simultaneously off also reinforce the weight on the connection between them will be presented in Chapters 2 and 3.

Architecture



$$\text{net} = b + \sum_i x_i w_i.$$

$$f(\text{net}) = \begin{cases} 1 & \text{if } \text{net} \geq 0; \\ -1 & \text{if } \text{net} < 0; \end{cases}$$

2.2.1 Algorithm

Step 0. Initialize all weights:

$$w_i = 0 \quad (i = 1 \text{ to } n).$$

Step 1. For each input training vector and target output pair, $s : t$, do steps 2–4.

Step 2. Set activations for input units:

$$x_i = s_i \quad (i = 1 \text{ to } n).$$

Step 3. Set activation for output unit:

$$y = t.$$

Step 4. Adjust the weights for

$$w_i(\text{new}) = w_i(\text{old}) + x_i y \quad (i = 1 \text{ to } n).$$

Adjust the bias:

$$b(\text{new}) = b(\text{old}) + y.$$

Note that the bias is adjusted exactly like a weight from a "unit" whose output signal is always 1. The weight update can also be expressed in vector **form** as

$$\mathbf{w}(\text{new}) = \mathbf{w}(\text{old}) + \mathbf{x}y.$$

This is often written in terms of the weight change, $\Delta \mathbf{w}$, as

$$\Delta \mathbf{w} = \mathbf{x}y$$

and

$$\mathbf{w}(\text{new}) = \mathbf{w}(\text{old}) + \Delta \mathbf{w}.$$

There are several methods of implementing the Hebb rule for learning. The foregoing algorithm requires only one pass through the training set; other equivalent methods of finding the weights are described in Section 3.1.1, where the Hebb rule for pattern association (in which the target is a vector) is presented.

2.2.2 Application

Bias types of inputs are not explicitly used in the original formulation of Hebb learning. However, they are included in the examples in this section (shown as a third input component that is always 1) because without them, the problems discussed cannot be solved.

Logic functions

Example 2.5 A Hebb net for the AND function: binary inputs and targets

INPUT			TARGET
x_1	x_2	1	
1	1	1	1
1	0	1	0
0	1	1	0
0	0	1	0

For each training input: target, the weight change is the product of the input vector and the target value, i.e.,

- 1- Identify how many training sample we should consider ?

What is mean training sample ? answer how many inputs are there ? x_1, x_2, x_3, x_4 4 training samples

Second question .. why we are considered training samples ?

Answer because on this training samples the iterations means how many times this algorithm will iterate it depends. so, 4 training samples means 4 iterations

- 2- Activation function (bipolar sigmoid $(-1 \text{ to } 1)$) if inputs are given bipolar $(-1 \text{ to } 1)$

Hebbian (how weights adjusted)

Set all weight and bias =0

For each training input: target, the weight change is the product of the input vector and the target value, i.e.,

$$\Delta w_1 = x_1 t, \quad \Delta w_2 = x_2 t, \quad \Delta b = t.$$

The new weights are the sum of the previous weights and the weight change. Only one iteration through the training vectors is required. The weight updates for the first input are as follows:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
			$(0 \ 0 \ 0)$
$(1 \ 1 \ 1)$	1	$(1 \ 1 \ 1)$	$(1 \ 1 \ 1)$

The separating line (see Section 2.1.3) becomes

$$x_2 = -x_1 - 1.$$

The graph, presented in Figure 2.7, shows that the response of the net will now be correct for the first input pattern. Presenting the second, third, and fourth training inputs shows that because the target value is 0, no learning occurs. Thus, using binary target values prevents the net from learning any pattern for which the target is "off":

The separating line (see Section 2.1.3) becomes

$$x_2 = -x_1 - 1.$$

The graph, presented in Figure 2.7, shows that the response of the net will now be correct for the first input pattern. Presenting the second, third, and fourth training inputs shows that because the target value is 0, no learning occurs. Thus, using binary target values prevents the net from learning any pattern for which the target is "off":

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
$(1 \ 0 \ 1)$	0	$(0 \ 0 \ 0)$	$(1 \ 1 \ 1)$
$(0 \ 1 \ 1)$	0	$(0 \ 0 \ 0)$	$(1 \ 1 \ 1)$
$(0 \ 0 \ 1)$	0	$(0 \ 0 \ 0)$	$(1 \ 1 \ 1)$

Example 2.6 A Hebb net for the AND function: binary inputs bipolar targets

INPUT	TARGET
$(x_1 \ x_2 \ 1)$	
(1 1 1)	1
(1 0 1)	-1
(0 1 1)	-1
(0 0 1)	-1

Presenting the first input, including a value of 1 for the third component, yields the following:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
			(0 0 0)
(1 1 1)	1	(1 1 1)	(1 1 1)

The separating line becomes

$$x_2 = -x_1 - 1.$$

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
(1 0 1)	-1	(-1 0 -1)	(0 1 0)
(0 1 1)	-1	(0 -1 -1)	(0 0 -1)
(0 0 1)	-1	(0 0 -1)	(0 0 -2)

However, these weights do not provide the correct response for the first input pattern.

Example 2.7 A Hebb net for the AND function: bipolar inputs and targets

INPUT	TARGET
$(x_1 \ x_2 \ 1)$	
(1 1 1)	1
(1 -1 1)	-1
(-1 1 1)	-1
(-1 -1 1)	-1

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
			$(0 \ 0 \ 0)$
$(1 \ 1 \ 1)$	1	$(1 \ 1 \ 1)$	$(1 \ 1 \ 1)$

The separating line becomes

$$x_2 = -x_1 - 1.$$

The graph in Figure 2.9 shows that the response of the net will now be correct for the first input point (and also, by the way, for the input point $(-1, -1)$). Presenting the second input vector and target results in the following situation:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
			$(1 \ 1 \ 1)$
$(1 \ -1 \ 1)$	-1	$(-1 \ 1 \ -1)$	$(0 \ 2 \ 0)$

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
			$(0 \ 2 \ 0)$
$(-1 \ 1 \ 1)$	-1	$(1 \ -1 \ -1)$	$(1 \ 1 \ -1)$

The separating line becomes

$$x_2 = -x_1 + 1.$$

The graph in Figure 2.11 shows that the response of the net will now be correct for the first three input points (and also, by the way, for the input point $(-1, -1)$). Presenting the last point, we obtain the following:

INPUT	TARGET	WEIGHT CHANGES	WEIGHTS
$(x_1 \ x_2 \ 1)$		$(\Delta w_1 \ \Delta w_2 \ \Delta b)$	$(w_1 \ w_2 \ b)$
			$(1 \ 1 \ -1)$
$(-1 \ -1 \ 1)$	-1	$(1 \ 1 \ -1)$	$(2 \ 2 \ -2)$

Even though the weights have changed, the separating line is still

$$x_2 = -x_1 + 1,$$

so the graph of the decision regions (the positive response and the negative response) remains as in Figure 2.11.

Binary i/p binary target is not correct to get AND gate as a target

Binary i/p bipolar target is not correct to get AND gate as a target

Bipolar i/p Bipolar target is correct to get AND gate as a target

Homework:

- 1- Hebb net , OR function , bipolar i/p binary target
- 2- Hebb net , OR function , Bipolar i/p , and target